

⑪ /dev

⇒ /dev stands for devices

It contains device file that represent hardware and virtual devices.

In linux everything is treated as file, including hardware

Why does /dev exists?

- Allows programs to communicate with hardware
- Provides file-based interface to devices
- Makes a hardware access consistent and secure

Example:

Writing to a device file → sending data to hardware
Reading from a device file → receiving data from hardware.

#* The "Device File" Concept

If you wanted to send data to your speaker in Linux, you wouldn't need ~~to~~ a fancy API. In theory, you could just "write" data ~~in~~ to a file in /dev that represent the speaker.

There are two type of file you will see:

- Character Device (c): Data is sent one character (one byte) at a time. Think of a keyboard or mouse.
- Block Device (b): Data is moved in large chunks (blocks). Think of your Hard Hard Drive or USB Stick.

Let's see difference between Character Device and Block device.

1) **lsblk** (The Block Devices Specialist)

If you want to understand Block Devices (`/dev/sda`, `/dev/nvme`, etc) this is the go to command. It ignores messy "character" devices (like mice) and only shows you the storage hardware.

It shows you a tree view of your harddrive, their partitions, and where they are mounted in your system.

Run:

lsblk

you will see:

```
loop0    7:0    0    55.5M    1    loop /snap/core...
```

```
⋮
⋮
⋮
```

```
sda      0    465.8G    0    disk
├─sda1   8:1    0    1G      0    part /boot/efi
└─sda2   8:2    0    464.7G    0    rom.
```


sda:
↳ This is your physical hard drive or SSD

sda1:
↳ This is your EFI partition. It's the "ignition switch" for your computer. When you turn on your computer, the motherboard looks here to find instruction on how to start Linux.

sda2:
↳ This is your Root partition (/). Everything (var, /usr/ etc) lives inside this slice of disk

loop:
→ You might see 30 extra loop. These are loop devices. They are not your physical hardware. In Ubuntu, snap packages (like, Firefox, Docker, VLC) are actually stored here as "compressed disk images".

When you run Firefox, Linux mount's the compressed file as if it were a physical drive so it can read the file inside

Every single line you see here is block device

- sda /dev/sda
- loop /dev/loop

If you run `ls -l /dev/sda`, you will see block at the beginning of the permissions, confirming block device.

Common Device files in /dev

1) The "Special" Device

These ~~are~~ files don't store data at your disk. They are "data stream" managed directly by linux kernel.

* /dev/null (The black hole)

⇒ Anything you send here disappears. It is used to silence annoying programs.

For example:

If you Run `ls /nonexistent_folder 2>/dev/`

Normally, the command would show an error. By redirecting the error to /dev/null, the error vanishes and your screen stays clean.

* /dev/zero

This file provides an infinite stream of null character (zeros)

Run:

```
head -c 100 /dev/zero | xxd
```

You will see


```
00000000:0000 0000 0000 0000 0000 0000 ---
:
:
:
000 00000:0000 0000 0000 0000 0000 0000 ---
```

So, basically the command ~~reads~~ reads the first 100 byte from /dev/zero (nothing) and shows them to you.

It's ^{used to} often created ~~to~~ empty file of specific size.

* /dev/random vs /dev/urandom.

These provide random numbers. /dev/urandom is the "unlocked" version and it is what 99% of people and we use.

Let's generate a password from /dev/urandom.

Run:

```
cat /dev/urandom 1 > /dev/null | tr -dc 'a-zA-Z0-9' |
fold -w 12 | head -n 5
```

You will have 5 line of random password of 12 word.

In old days of linux, there was a big difference between two.

- /dev/random: It only gives you random number when your PC has enough "entropy" (a physical noise from your keyboard, ~~noise~~ mouse, hardware)

It only gives
If you ask for 100 random numbers and if there is no enough entropy say it has 50 then, it blocks (stop your computer) and makes you wait until noise is generated.

- `/dev/urandom`:
The u stands for unlimited. It uses a fancy math formula to generate random number. It generates random number even if no enough entropy is there.

Why do we use `/dev/urandom` more?

⇒ Modern Math has proven `/dev/urandom` is just as secure as almost everything including (password and encryption) and it never makes your computer wait.

What are `/dev/urandom` & `/dev/random` used?

We used manually to make password. But Linux ~~key~~ system uses hundred times a second without telling you.

① Encryption:

Every time you visit a website (Google, Reddit) your browser generate a secret key so hacker can see your data. It ask `/dev/urandom` to generate random number to start encryption.

② ASLR (The Security ~~State~~ Shuffle)

To prevent hackers from predicting where a program is in RAM. It shuffles memory location every time app. It use `/dev/urandom` to decide the shuffle order.

③ The UUID

When you create a new file or disk partition a unique ID like `550e8400-e29b-43d4-a716-4466544000`. The random numbers are generated from `urandom`, to ensure no two files have same ID.

Fun Command:

To see how much "Random Power" your computer currently has stored up, run this command.

```
cat /proc/devsys/kernel/random/entropy-  
avail
```

You will see a number

If you see number $> (256 \text{ or } 4096)$:

Your system is noisy enough to be very secure

In Modern Linux it's bit different

First, let's check which one is faster

Run:

```
dd if=/dev/urandom of=/dev/null  
bs=1M count=10
```


and

`dd if=/dev/random of=/dev/null bs=1M count=10`

You won't find much difference in speed
Because in modern CPUs they have a
feature called RDRAND - tiny piece of hardware
inside your processor that generate random
numbers at incredible speed. Linux uses this
to keep the 'tank' full constantly.

2) The terminal devices

* ~~/dev~~

These files are how linux handles your typing
and displaying

* `/dev/tty`

This always represents a terminal you are
currently using.

• `Run`

```
echo "Hello me" > /dev/tty
```

It prints Hello me right back at your screen.
It's a way for a script to talk to user
even when script's output is being sent some
where else.

↳ /dev/pts (Pseudo-Terminal)

When you open up a window terminal (GNOME Terminal) or connect via SSH, ~~you~~ you are assigned slave terminal like /dev/pts/0

Command:

who

tells / shows you which pts number you are currently using

For example:

alish	seat 0	(login screen)
alish	tty 2	(tty 2)

seat 0: represents physical hardware you are touching - your monitor, mouse

login screen: This tells us that graphical desktop (GNOME) most likely took control of physical seat

tty 2: In linux, your graphical desktop usually runs on specific "virtual console". Usually tty 1 for boot text and tty 2 or tty 7 for desktop

If you run tty you will see /dev/pts/0
so this is the slave pts of your current terminal.

You might get confuse that wait I am typing in a terminal window why no /dev/pts/0 is showing.

Well we know we can open 20 different window. Each gets pts.

Run

tty

It won't say tty2 you will see /dev/pts/0

So, tty2 → parent session

/dev/pts/0 is → child (window you are typing in)

Fun Command. (Very)

- Open another window Terminal
- Its pts number would be /dev/pts/1
Type:

```
echo: "Hi I am me from next window" > /dev/pts/0
```

In your /dev/pts/0 you will see echo message.

When you run `ls /dev`, you will see many files, folders, that you usually ignore unless you are kernel developer.

For example .

* `VCS` and `VCSA` : These are "Virtual Console Screen" that manage the text only console (the one you see if your graphic card fails) You almost never touch them

* `mcelog` : This is Machine Check Exception . It's a Mailbox for hardware error message from CPU, if it is important you'll usually know your computer is crashing

* `hugepages` : This is a memory management features for high performance database.

12) /proc

/proc is the virtual filesystem that provides

- Kernel Information
- System information
- Process information

It does not store real file ~~in~~ on disk

It is created in memory (RAM) by the kernel.

Why /proc exist

linux exposes internal system data as file

Instead of special command you can simply

`cat /proc /yourfile name`

Inside /proc

We can categorize into two:

① System Information Files

- `cat /proc/cpuinfo` : Gives every detail about your processor
- `cat /proc/meminfo` : A massive, detailed on how your RAM is being used.
- `cat /proc/uptime` : How many seconds your computer ~~is~~ has been on since last reboot.

- `cat /proc/` ~~version~~ version: Exactly which version of the Linux kernel you are running

② Process ~~directories~~ Directories

Inside `/proc` you will see numbers:

`ls /proc`

For e.g.:

1

10

1059

on

Every app running on your computer (Firefox, system clock, terminal) has its own folder here

Every running process gets its own directory.

Run:

`echo $$`

(You printed your shell ID)

For e.g. you got 16941

`cat /proc/16941/status`

(I will get internal process about this process)

③ Special Files

- `/proc/self`

It always points to the folder to whichever program is looking at it.

Run: ~~to /pro~~ `cat /proc/self/status`

You will see information about cat command.

- `/proc/cmdline`

→ Show kernel boot parameter

- `/proc/sys`

Some files in `/proc` specially (`/proc/sys`) allow us to change how linux behave without restarting.

Example: You can change your hostname of your machine without rebooting.

Run:

`sudo hostname new_name`

This change is actually happening in your `/proc/sys/kernel/hostname`.

Why /Proc is Important?

- ⇒ Most of the Linux command we see is just a fancy reader for the /proc Folder.
- When you run `free -h` it is just reading from `/proc/meminfo`
- When you run `uptime`, it is just reading from `/proc/uptime`.

13) /sys

let's us

While /proc mostly read information, /sys lets
up change hardware setting on fly the fly.

You can dim your brightness, turn off your
CPU core, eject a disk by writing values to files
here.

→ Just like /proc it is created in RAM

Run `ls /sys`

You will see directories:

- /sys/class
- /sys/bus
- /sys/power
- /sys/firmware firmware
- /sys/hypervisor
- /sys/kernel
- /sys/block
- etc

* `/sys/block`

→ It is a shortcut to all your block devices

Run:

`ls /sys/block`

ee' loop0 ... loop30
sda sro

sda = Hard disk or SSD

sro = Optical drive (CD/DVD)

* /sys/class

⇒ It manages groups devices by type (e.g net, input, backlight)

Run:

```
Cat /sys/class/net/eth250/  
operstate
```

You will see network interface status
up or down.

* Run

```
Cat /sys/class/power-supply/BAT1/capacity
```

You will see percentage of you battery.

* /sys/kernel

⇒ Here, are kernel parameters and runtime configuration

One of the important of /sys/kernel subdirectory
is debug.

Developer knows / can see exactly what hardware
drivers are doing. It's usually locked up for
security reason but it's the ultimate debugger
for linux pro.

* /sys/devices

One of the most important folder in /sys directory. It is the root of the tree

Everything that is in /sys is symbolic link that points to /sys/devices.

- /sys/devices: This is the actual, physical map of how your hardware is plugged to your motherboard. It follows the electrical path (Bus → Bridge → Controller → Device).
- /sys/class → This is the filtered view for humans. Instead of following cables. It just groups things by what they are.

When and How /sys/device works?

If you run ls /sys/device, you'll see thing like pci0000:00. Inside those there are even longer files/name. This is because kernel is tracking exact address on your hardware bus.

Example:

If you have a ~~ttt~~ USB mouse, it's True path might be:

/sys/device/~~/~~pci0000:00/0000:00:1f.2

That's a lot to type so Linux creates a shortcut for you at:

`/sys/class/input/mouse0`.

^{only}
You usually ~~don't~~ go into "Deep Device" folder when you are doing advanced hardware troubleshooting.

Why do we need `/sys`?

Before `/sys` exist, the `/proc` folder was getting very messy. It was full of hardware data that didn't ~~bel~~ belong to process folder. Developer created `/sys` to be clean, organized map of the computer's physical structure.

`/proc`: What are apps doing? (Dynamic, messy, Read-only)

`/sys`: What is the hardware doing? (Structured, clean, read/write)

14) /mnt

/mnt stand for mount

Concept of mounting

In windows:

Hard Disk $\rightarrow$ C:

USB Drive $\rightarrow$ D:

DVD $\rightarrow$ E:

Each device get its own separate drive letter.
So, storage looks like

C:\

D:\

E:\

They are separate

In linux, we don't use drive letter.

Linux has only one main tree which is "/"

Everything must connect somewhere inside /.

In simple word:

Take a storage device and attach it
to a folder inside /.

Simple Example:

Suppose you Plug in USB drive

Linux detects it as:

`/dev/sdb1` or `/dev/sda1`

By But you cannot use it yet.

You must attach it somewhere like:

`/mnt`.

To mount we run:

`sudo mount /dev/sda1 /mnt`

Now,

`/mnt`

contains the USB file

So, instead of drive letter like (D:/, E:/) we use `/mnt`.

More clear example:

Let's say that USB has photo you want access

So, you insert the USB to port

Linux Detects it:

`/dev/sda1`

But at this point:

You cannot see the picture

You cannot open file

It's just a raw device

Now you mount:

```
Sudo mount /dev/sda1 /mnt.
```

Now, USB filesystem is attached to /mnt.

Now,

```
ls /mnt.
```

You will see

photo1.jpg

photo2.jpg

DCIM/

Videos/

Now you can see:

- Open photos, copy file etc

In Linux, there is another directory that does you same work automatically.

media

well we are using Desktop Version of Linux.
Our computer wants to be helpful.

- 1) You plug in the USB
- 2) A program called udisks sees it and say 'Oh! user just plugged in disk.'
- 3) It automatically hooks it to /media/user/USBname
- 4) A window pops up showing your photo.

Why use /mnt then?

If /media is automatic, why does /mnt exist?

You use it when you want full control.

Example:

→ Repairing a broken system:

If your computer won't boot and you use a "Live USB" to fix it, you would mount your broken hard drive to /mnt to rescue & your file

- Permanent storage: If you have a secondary drive inside your PC that stays there forever, you might mount it to /mnt / storage so it always in the same place.

~~Scenario~~ Scenario 1: Resetting a Forgotten Password

If you are locked out of your computer, you can't log in to change password.

Then,

- ① Boot from a live USB
- ② Open terminal and find your hard drive using `lsblk`.
- ③ Mount it: `sudo #mount /dev/sda1 /mnt` you are moving harddisk to live usb.

- (4) You use command `chroot /mnt` tricks the computer into thinking `/mnt` is root.
Now, if you type `passwd user`, it changes the password on the broken drive, not `usb`.